

在原有DNN推理任务部署模型中引入能耗建模与能耗优化目标——修改方案

1 符号与参数

- P_{cpu}^n : 计算节点 n 处理推理任务时的动态计算功率 (W)。
- P_{tx}^l : 在网络链路 l 上发送数据的传输功率 (W)。
- E_{limit}^i : DNN 推理任务 $task_i$ 能够容忍的最大能耗约束。
- E_{des}^i : DNN 推理任务 $task_i$ 的期望能耗（用于 DSSA）。

2 Energy Model

在原论文的延迟模型 (Delay model) 和可靠性模型 (Reliability Model) 之间，加入能耗模型 (Energy Model)。

1. 计算能耗建模

某一层 v_j^i 如果部署在节点 n 上，其计算能耗等于处理延迟乘以该节点的计算功率。整个任务的计算能耗为：

$$e_{proc}^i = \sum_{n=1}^{|N|} \sum_{j=1}^{|V_i|} x_{i,j}^n * d_{i,j}^n * P_{cpu}^n$$

2. 传输能耗建模

层与层之间的数据依赖 e_j^i 若在链路 l 上传输，其传输能耗等于传输延迟乘以发射功率。总传输能耗为：

$$e_{tran}^i = \sum_{l=1}^{|L|} \sum_{j=1}^{|E_i|} y_{i,e_j^i}^l * d_{e_j^i}^l * P_{tx}^l$$

3. 总能耗建模（加入这个作为第四个优化目标）

$$E_{all}^i = e_{proc}^i + e_{tran}^i$$

3 Algorithm Design

3.1 RDODA

- 适应度计算 (Fitness Calculation): 在评估种群时，额外计算个体的 E_{all}^i 。如果个体违反了能耗约束 ($E_{all}^i > E_{limit}^i$)，像处理延迟约束一样，将其能耗适应度设为极大的负向惩罚值，确保其被淘汰。
- 非支配排序逻辑更新 (Eq.14):
非支配排序需要增加能耗比较。

3.2 DSSA

- 期望权重计算：引入能耗期望 E_{des}^i ，计算能耗的相对权重 I_e :
$$I_e = \frac{E_{max} - E_{des}^i}{E_{max} - E_{min}}$$
- 归一化比例：引入能耗比重，分配给四个目标的比重变为：
$$I'_k = \frac{I_k}{I_1 + I_2 + I_3 + I_e}, \quad k \in \{1, 2, 3, e\}$$
- 最优选择公式：将挑选最符合期望的部署方案公式扩展为四项加权和。

算法的定制化设计

RDODA本质上是对经典NSGA-II的直接应用（两点交叉 + 随机变异），缺乏针对DNN拓扑依赖或边缘网络特性的定制设计，创新性不足。

1 针对DNN拓扑

Na J, Zhang H, Lian J, et al. Partitioning dnns for optimizing distributed inference performance on cooperative edge devices: A genetic algorithm approach[J]. Applied Sciences, 2022, 12(20): 10619.

论文将 DNN 分区（**Partitioning**）与分布式部署 问题建模为一个带约束的优化问题。其核心目标是在保证资源受限的边缘设备能够运行的前提下，最小化整个 DNN 推理的总延迟。

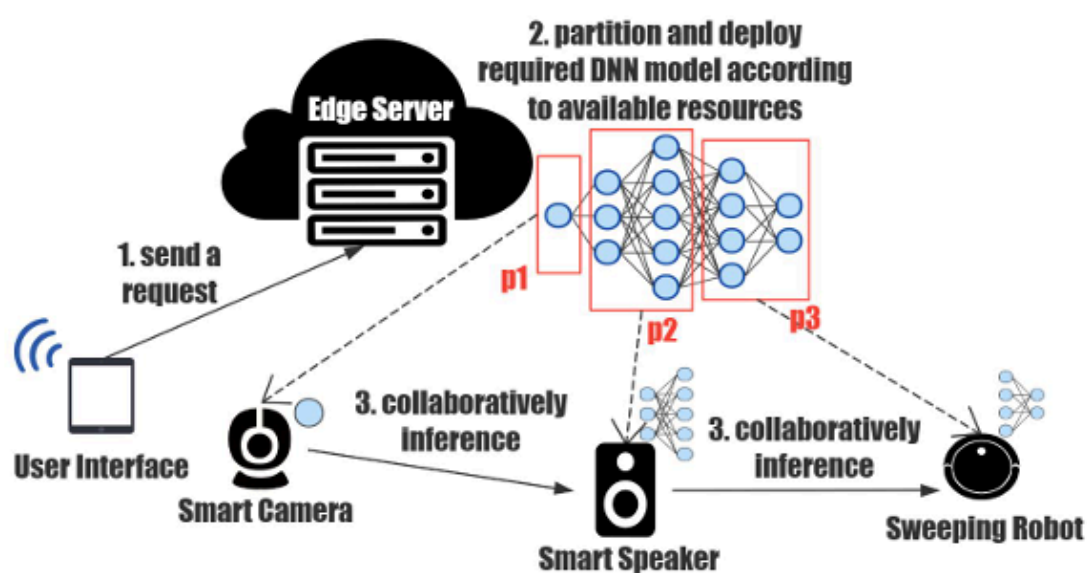


Figure 1. Illustration of a distributed DNN inference by collaboration between multiple edge devices.

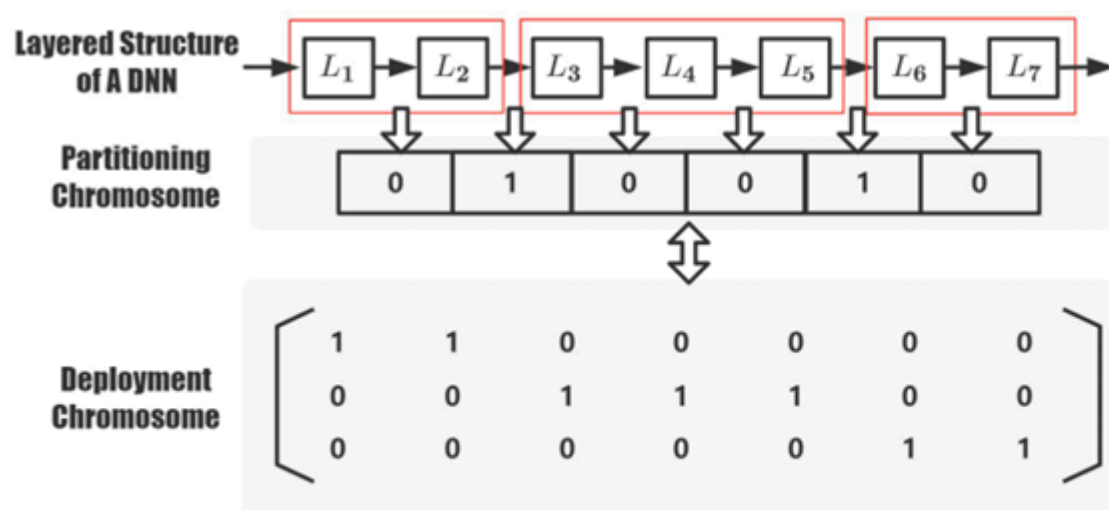


Figure 3. An example the relationship between a DNN structure, a partitioning chromosome and a deployment chromosome.

DNN 总层数 l 是由模型本身固定的，不同层可以被分配到同一个物理设备进行计算。层与层之间是串行连接的，即“链式结构”。

例如：模型有 10 层（ $l = 10$ ），你可以决定在第 2 层后面切一刀，在第 5 层后面切一刀。

这种“切法”就是论文中提到的 **Partitioning Chromosome**（分区染色体）。它的长度是 $l - 1$ ，每一个位置如果是 1 就代表切开，是 0 就不切。

块的产生：一旦你定下了切分点，这 l 层就被划分成了 n 个“块”（sub-models）。比如你决定把 1-3 层给设备 A，那么 1-3 层就合在了一起。

论文的建模强制要求了执行过程是串行的：

1. 设备 1 必须跑完它负责的那几层（块 p_1 ）。
2. 设备 1 把结果传给 设备 2。
3. 设备 2 收到数据后才能开始跑它负责的那几层（块 p_2 ）。

不能并行：在这种建模下，设备 2 不能和设备 1 同时跑，因为设备 2 必须等设备 1 的输出作为它的输入。

且这篇论文约束：每一块唯一地部署到一个特定设备上：

- 反面例子（基础 GA 可能犯的错）：
基础算法可能会把第 1、3、5 层分给设备 A，把 2、4、6 层分给设备 B。
 - 虽然设备 A 跑了三层，但这三层不连续。
 - 结果： $1(A) \rightarrow 2(B) \rightarrow 3(A) \rightarrow 4(B) \rightarrow 5(A) \rightarrow 6(B)$ 。
 - 代价：数据在 A 和 B 之间来回传了 5 次！
- 论文的做法（改进 GA）：
它强制要求你只能切 $n - 1$ 刀。
 - 比如切在第 3 层和第 6 层后面。
 - 结果：1,2,3 $\rightarrow$ 4,5,6 $\rightarrow$ 7...l。
 - 优点：数据只在 $A \rightarrow B$ 和 $B \rightarrow C$ 传了 2 次。

建模比较

维度	第一篇 (Na Jun 等)	第二篇 (Wang Ying 等)
模型结构	线性链式 (Linear Chain)	有向无环图 (DAG)
部署映射	1对1 (一块对应一个设备)	多对1 (多个层可对应一个设备)
并行能力	完全串行 (必须等前一个设备传数据)	支持并行 (不同分支可同时在不同设备跑)
时延计算	简单的各块累加	寻找 DAG 的关键路径 (Max Path)

在 Na Jun 的这篇论文中，算法的核心创新点在于提出了“双层染色体结构”。

解决的具体问题（痛点）：传统的遗传算法在处理此类优化时，通常直接对“部署矩阵”（即哪一层在哪台机器上）进行随机交叉和变异。这会导致两个严重后果：

- 层不连续（Non-contiguous layers）：算法可能会把第 1、3 层分给设备 A，第 2 层分给设备 B。
- 数据回传（Data bounce-back）：在执行时，数据流向变成：设备 A (层1) $\rightarrow$ 设备 B (层2) $\rightarrow$ 传回设备 A (层3)。
- 无效搜索：基础 GA 会生成大量这类“愚蠢”的解，浪费了海量的计算时间去评估这些明显不合理的方案，导致收敛极慢。

为了解决上述问题，作者将“染色体”拆分为两个逻辑层，这是该论文最主要的创新：

第一层：分区染色体（Partitioning Chromosome）

- 形式：一个一维向量（例如：`[0, 1, 0, 0, 1, 0]`）。
- 作用：它只决定“在哪里剪开模型”。`1` 代表切分点，`0` 代表不切。
- 改进点：交叉和变异操作只在这一层进行。因为无论你怎么交叉或变异这个一维向量，切出来的每一段永远是物理连续的层。这从根本上杜绝了“数据回传”的可能性。

第二层：部署染色体（Deployment Chromosome）

- 形式：一个 $n \times l$ 的矩阵。
- 作用：决定切好的“块”分别扔给哪台设备跑。
- 关联：论文设计了两个算法（Algorithm 1 和 2）用于这两层染色体之间的转换。

具体改进操作

- 种群初始化：先随机生成“切分点”，再生成对应的“部署方案”，确保初始种群全都是有效解。
- 交叉与变异逻辑：
 - 当两个父代交换基因时，它们交换的是切分位置，即只在分区染色体层面进行交叉和变异。这样做的好处是：无论如何变异，切分出来的子块永远是物理连续的层，保证了生成解的有效性。
 - 这种设计大幅缩小了搜索空间（Search Space），因为算法不再需要去尝试那些“不连续”的无效组合。
- 适应度函数（Fitness Function）的优化：
 - 论文在计算适应度时，不仅考虑总时延，还把内存限制和电量限制设为“死线”。不满足约束的解会被打出极低的适应度分数（ 10^{-6} ），确保进化方向始终朝着可行解靠近。

Ye C, Zhang R, Li X, et al. HGA-DP: Optimal Partitioning of Multimodal DNNs Enabling Real-Time Image Inference for AR-Assisted Communication Maintenance on Cloud-Edge-End Systems[J]. Information, 2025, 16(12): 1091.

关注的是多模态 DNN（Multimodal DNNs）在“云-边-端”三级架构下的部署。

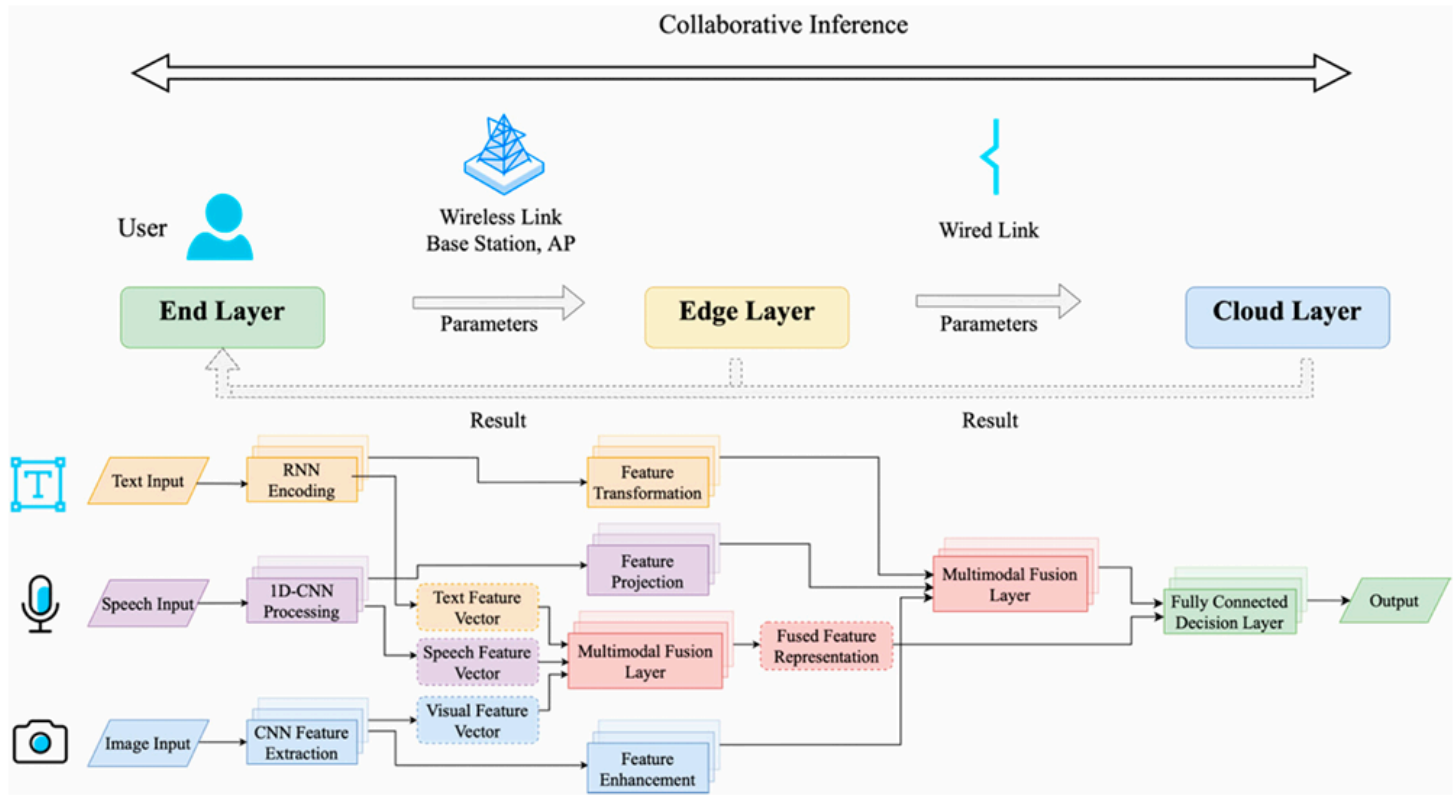


Figure 1. Multimodal inference workflow in Cloud-Edge-End architecture.

虽然简单的神经网络可以被概念化为层的线性序列，但这种表示方法不足以描述现代多模态模型。我们将多模态应用建模为一个有向无环图（Directed Acyclic Graph, DAG）。

将DAG中的节点分组为有意义且结构内聚的块。这将问题从对微观操作的复杂图进行分区，转变为对宏观模块的简化图进行调度。

- 原始 DAG 模型： 将多模态 DNN 建模为有向无环图 $G = (V, E)$ ，每个节点 v_i 是一个算子，边 e_{ij} 是数据依赖。
- 粗粒度分群 DAG (GDAG)： 为了降低搜索空间，并保护多模态融合块（Fusion Blocks）不被拆散，作者将细粒度的算子聚合成“功能子图”（Subgraphs/Macro-modules）。
 - 语义完整性： 强制将跨模态注意力机制（Cross-modal Attention）等核心融合层划分为不可分割的单元。
 - 建模结果： 最终处理的对象是 $G' = (S, E')$ ，其中 S 是功能子图的集合。

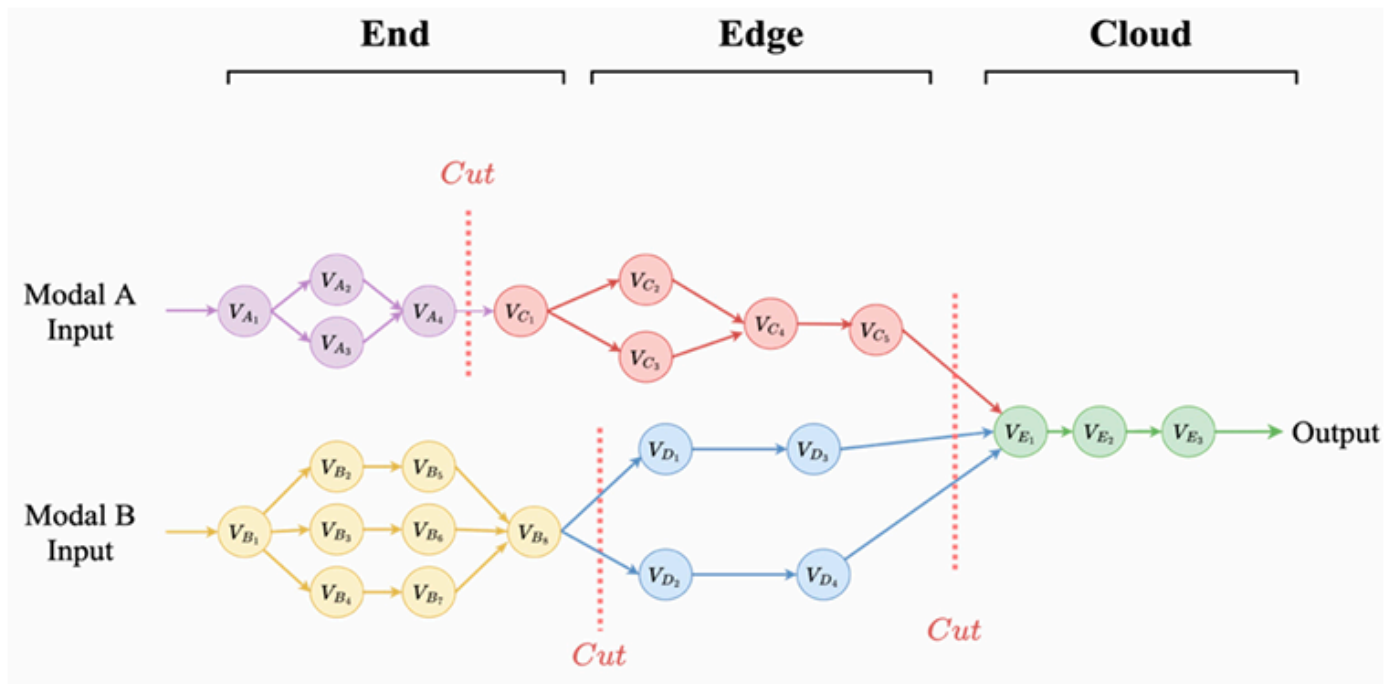


Figure 2. GDAG-based partitioning of the model DAG.

约束：数据只能从“端 → 边 → 云”单向流动，严禁数据逆流（例如数据传到云端处理后再传回终端跑下一层）。这确保了推理的逻辑顺序符合层级架构。

优化目标：延迟，能耗（只考虑用户终端设备的能耗）。

1.1 染色体设计与适应度函数

- 染色体编码： 算法将部署方案编码为一个整数向量 $S = [s_1, s_2, \dots, s_N]$ 。
 - 每个基因 s_k 的取值范围是 $\{0, 1, 2\}$ ，分别代表终端（0）、边缘（1）和云端（2）。
- 适应度函数（评价标准）： $Cost = w_T \cdot (\text{归一化延迟}) + w_E \cdot (\text{归一化能耗})$ 。

1.2 进化算子

- 选择： 采用锦标赛选择法（每次随机抽 5 个方案，选最好的），保证了种群的竞争性。
- 交叉： 采用单点交叉，随机切开两个“父母”的方案并互换，这有助于保留局部优秀的部署块。
- 变异： 引入了动态变异率。在算法开始阶段变异率高（鼓励到处乱撞找新方案），随着迭代次数增加变异率降低（鼓励在好方案附近精细打磨）。

1.3 约束感知修复机制 (算法创新设计)

这是这篇论文解决“非法解”的关键。在遗传算法的随机交叉和变异过程中，经常会出现“数据逆流”的错误方案（例如：第 1 层分到了云端，第 2 层却分到了终端）。

- 规则约束： 强制要求 $s_i \geq \max(\text{predecessors})$ 。即任何一个任务节点的部署层级，必须大于或等于它所有前驱节点的层级。
- 修复过程（算法 1）：
 - 按照拓扑排序遍历所有子图。
 - 检查当前节点的层级是否低于其前辈。
 - 如果是，则强行提升当前节点的层级，直到它合法为止。
- 意义： 这个机制保证了种群中所有的个体都是“合法”的（数据只从端流向边，再流向云）。

1.4 混合邻域搜索 (算法创新设计)

- 局部优化（算法 2）： 在每一代进化结束后，算法会拎出当前最优秀的个体（精英方案）。
- 邻域探索： 对这个精英方案的每一个基因进行微调（比如把某个任务从边缘改到终端，或者从边缘改到云端，即 ± 1 操作）。
- 贪心替换： 如果微调后的方案比原来更好，就直接替换掉种群中的弱者。
- 意义： 这种“全局搜索 + 局部微调”的组合，让算法收敛速度极快，且不容易陷入局部最优。

Wang P, Ouyang T, Liao G, et al. Edge intelligence in motion: Mobility-aware dynamic DNN inference service migration with downtime in mobile edge computing[J]. Journal of Systems Architecture, 2022, 130: 102664.

多出口DNN（Multi-exit DNN）

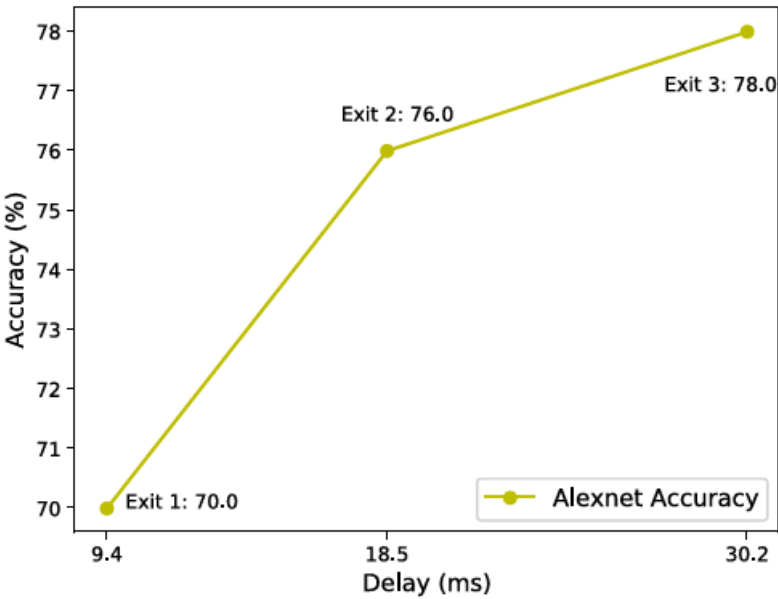


Fig. 1. Inference delay and accuracy of AlexNet with 3 exit point.

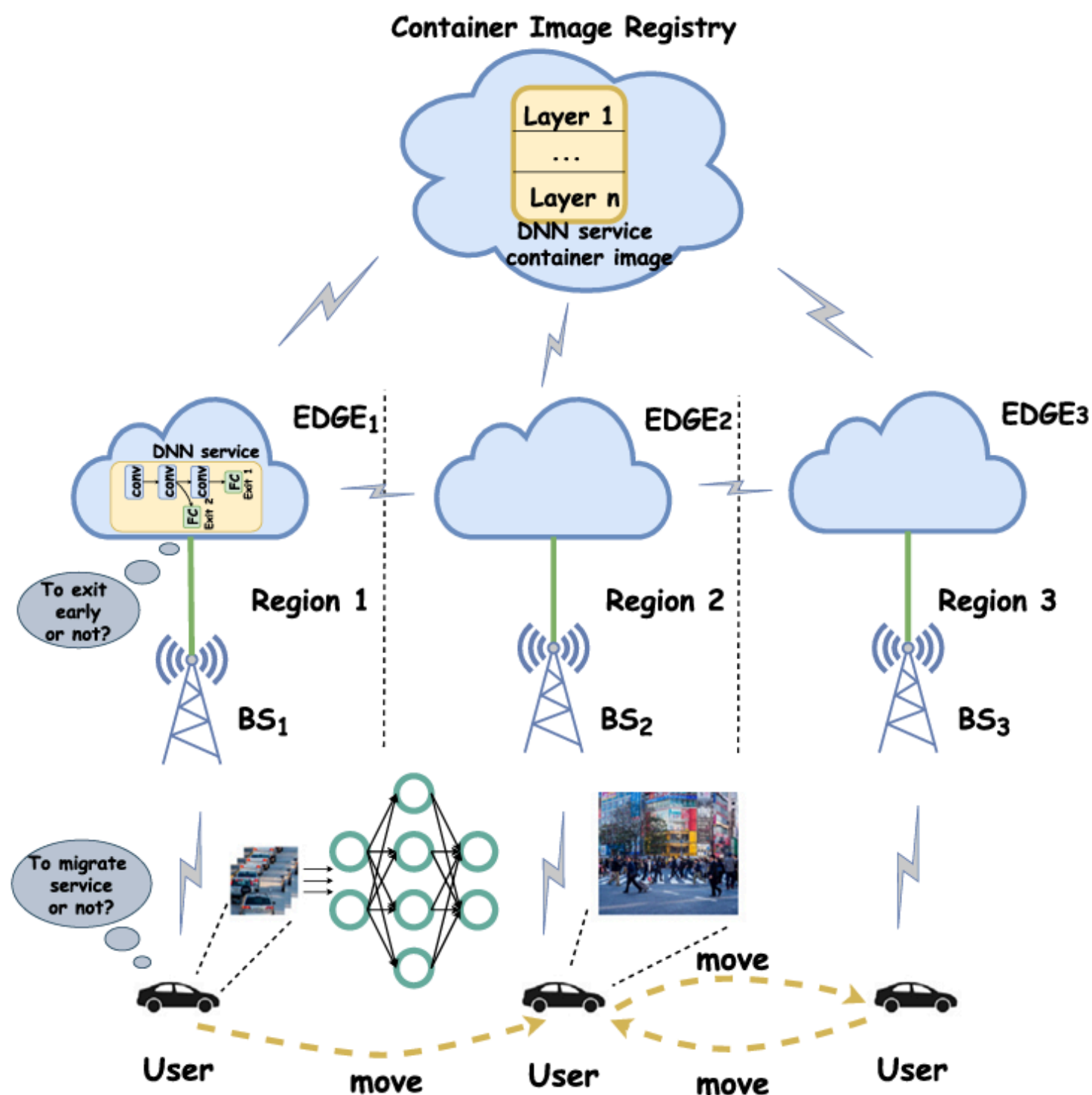


Fig. 2. Dynamic DNN service migration in mobile edge computing.

如果说传统的 DNN 是一列“直达列车”（必须坐到终点站才能下车看到结果），那么多出口 DNN 就是一列“有多个停靠站的通勤火车”（你可以在中途任何一个站下车，提前拿到一个虽然不那么完美、但可用的结果）。

- 传统模型：输入 → 第 1 层 → ... → 第 100 层 → 输出结果。
- 多出口模型：
 - 输入 → 第 10 层 → 出口 1（得到初步结果）
 - (继续计算) → 第 40 层 → 出口 2（得到更准的结果）
 - (继续计算) → 第 100 层 → 主出口（得到最准的结果）

1.4.1 A. 样本有难易之分（早退机制 Early Exit）

- 简单样本：比如识别一张背景干净、特征明显的“狗”的照片。模型可能在第 5 层就已经有 99% 的把握它是狗了。这时候让他跑完 100 层就是纯纯的浪费算力。
- 困难样本：比如一张模糊、光线暗、狗还躲在草丛里的照片。模型需要跑完 100 层，利用深层的复杂特征才能确定。

1.4.2 B. 环境有快慢之别（适应性推理 Adaptive Inference）

这正是你读的那篇中山大学论文（Wang Pu 等）关注的点：

- 网络极好：既然传输不花时间，我就让模型在边缘服务器上跑完，给我一个 100% 准确的结果。
- 正在搬家（迁移）/信号极差：此时边缘服务器没法用，或者数据传不过去。为了不让系统“卡死”，我干脆在手机本地只跑前几层，从出口 1 拿个 70 分的结果。虽然不完美，但总比没有强。

2 针对边缘网络

Chen M, Qi P, Chu Y, et al. Genetic algorithm with skew mutation for heterogeneous resource-aware task offloading in edge-cloud computing[J]. Heliyon, 2024, 10(12).

为了提升GA的性能，我们提出利用一种偏斜变异算子（skew mutation operator），在种群进化过程中，该算子将基因与资源异构性相关联。

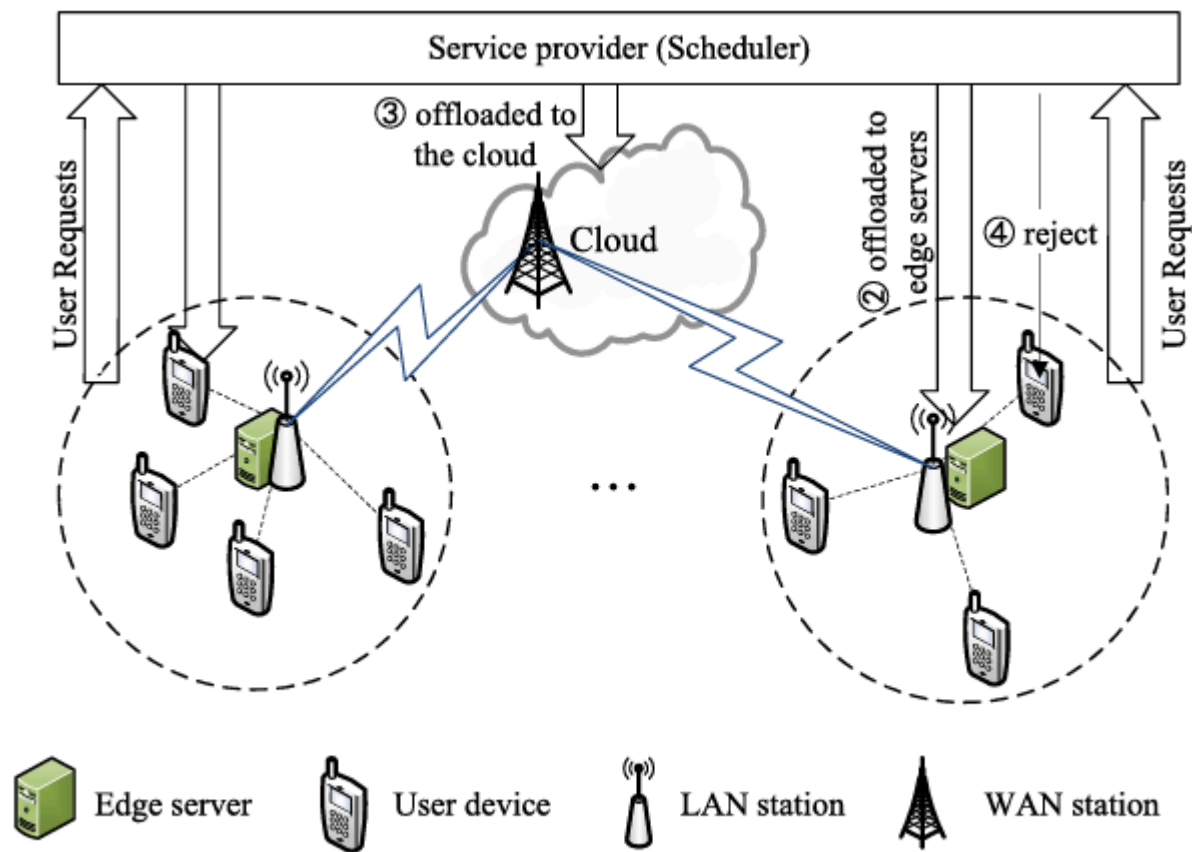


Fig. 1. The skeleton of the edge-cloud and possible processing manners of requests.

这篇论文提出的算法被命名为 **SkewGA**（带偏斜变异的遗传算法）。它在传统遗传算法（GA）的基础上，针对边缘-云（Edge-Cloud）异构资源的特性，做了两项核心改进。

2.1 1. 核心算法流程

SkewGA 遵循遗传算法的标准骨架，但在评估和变异这两个关键环节进行了重构：

1. 编码：采用整数编码，染色体上的每个基因代表一个任务，基因的值代表分配给该任务的特定计算核心（Core）索引。
2. 初始化：随机生成初始种群。
3. 适应度评估 + 重调度：计算每个方案的任务完成数和资源利用率。
4. 选择：采用精英保留策略（前10%直接晋级）+ 轮盘赌选择。
5. 交叉：采用均匀交叉（Uniform Crossover）。
6. 变异（创新点）：采用偏斜变异（Skew Mutation）。

2.2 2. 算法创新点一：偏斜变异算子 (Skew Mutation)

这是本论文最主要的创新。

- 传统 GA 的问题：传统的变异是“盲目”的。如果一个基因发生变异，它会以相等的概率随机选择任何一个资源。在异构环境中，这会导致任务被错误地分配到算力极弱或负载极高的节点上，从而导致任务超时被拒绝。
- SkewGA 的改进逻辑：变异不再是随机“乱撞”，而是向算力强或空闲多的资源“偏斜”。
- 数学实现（Eq 23）：
算法引入了松弛时间（Slack Time）的概念：
$$P_k = \frac{\max\{0, d - f_k\}}{\sum \max\{0, d - f_k\}}$$
 - d 是任务的截止时间。
 - f_k 是任务在核心 k 上运行的预期完成时间。
 - $d - f_k$ 就是“松弛时间”，表示这个核心离“超时边缘”还有多远。
- 效果：某个核心处理该任务越快（松弛时间越长），它被变异选中的概率就越高。这使得算法能够自动感知节点的异构性，将任务引导至更合适的资源，显著提高了负载均衡能力。

2.3 3. 算法创新点二：任务重调度机制 (Task Rescheduling)

这是为了弥补启发式搜索随机性而设计的二次优化环节。

- **痛点：** 遗传算法在经过交叉和变异后，生成的个体往往不完美，可能存在某些任务因为零点几秒的偏差就错过了截止时间（被标记为 **Rejected**）。
- **改进操作（算法流程图 Fig 3）：**
在每一代评估适应度之前，算法会执行一个特殊的“抢救”逻辑：
 1. 提取出当前方案中所有被拒绝（**Rejected**）的任务。
 2. 使用 **First Fit**（最先匹配）策略，扫描所有资源核心。
 3. 如果发现某个核心有足够的空闲碎片时间能塞下这个任务且不影响其他任务，就强行把该任务插进去。
- **效果：** 这种“贪心微调”能最大化榨取节点的剩余价值，进一步提升了任务完成率（**QoS**）。

Kumar M S, Raha A, Das D, et al. SlimEdge: Lightweight Distributed DNN Deployment on Constrained Hardware[J]. arXiv preprint arXiv:2512.22136, 2025.

3 针对违反延迟约束的个体赋负值

Deb的约束支配原则 (Constraint-Dominance Principle, CDP)：修改“支配（**Dominance**）”的定义。在比较两个个体 i 和 j 时，遵循以下规则：可行解永远优于不可行解；两个不可行解比较时，约束违反度（**CV**）小的胜出。

Deb K, Pratap A, Agarwal S, et al. A fast and elitist multiobjective genetic algorithm: NSGA-II[J]. IEEE transactions on evolutionary computation, 2002, 6(2): 182-197.

ToP (Two-Phase Framework) 框架：

Liu Z Z, Wang Y. Handling constrained multiobjective optimization problems with constraints in both the decision and objective spaces[J]. IEEE Transactions on Evolutionary Computation, 2019, 23(5): 870-884.

这是近年来在约束多目标优化（**CMOP**）领域非常流行的框架。

- **第一阶段 (Phase 1)：** 通过将多目标问题（**CMOP**）简化为单目标问题（**CSOO**），忽略目标间的竞争，利用“单点突破”的能力快速穿透复杂的决策空间约束。
- **第二阶段 (Phase 2)：** 恢复多目标属性，利用第一阶段已经进入可行域或靠近 **Pareto** 前沿（**PF**）的种群，快速向两侧扩散，补全整个解集。

3.1 第一阶段：约束单目标优化 (CSOO)

这一阶段的目标是快速找到“有希望的可行区域”。

3.1.1 目标函数转换

将原有的 m 个目标函数 $f_1, f_2, \dots, f_m$ 转换为一个单目标函数 $f'(x)$ ：
$$\min f'(x) = \sum_{i=1}^m f_i(x)$$

原理： 这种等权重求和方式对应的解通常位于 **Pareto** 前沿的中心位置。从中心点出发向四周扩散，比从边缘扩散更高效。

3.1.2 搜索引擎（**DE** 差分进化）

论文中推荐在第一阶段使用差分进化算法（**Algorithm 2**），因为它在处理复杂单目标约束问题上非常强劲。

策略混合： 以 0.5 的概率交替使用 **DE/current-to-rand/1**（保持旋转不变性）和 **DE/rand-to-best/1/bin**（利用当前最优解加速收敛）。

3.1.3 约束处理：可行性规则 (Feasibility Rule)

在比较两个个体 x_u 和 x_v 时，遵循以下准则：

- 1. 如果两个都可行，选 $f'(x)$ 较小的那个。
- 2. 如果一个可行一个不可行，选可行的那个。
- 3. 如果两个都不可行，选约束违反度 (CV) 较小的那个。

3.2 关键节点：第一阶段到第二阶段的转换条件

这是 ToP 框架最精妙的地方。算法不会一直运行单目标搜索（否则种群会退化为一个点），而是当满足以下 两个条件同时成立 时，立即切换到第二阶段：

3.2.1 条件 1：可行解比例 (Feasibility Proportion)

$P_f > 1/3$

- 含义：种群中至少有 1/3 的个体已经进入了可行域。这保证了第二阶段有足够的“种子”来铺开搜索。

3.2.2 条件 2：种群收敛程度 (δ)

- 1. 归一化处理：对种群中所有可行解 y_i ，计算各维目标的归一化值 $\bar{f}_j(y_i)$ 。
- 2. 求和：计算每个解的归一化目标总和 $\bar{f}'(y_i) = \sum \bar{f}_j(y_i)$ 。
- 3. 排序与差值：将可行解按 $\bar{f}'$ 排序，计算前 1/3 可行解中，最大与最小 $\bar{f}'$ 的差值，记为 δ 。
- 4. 判断：如果 $\delta < 0.2$ ，则认为种群在当前可行域内已经足够收敛。

3.3 第二阶段：约束多目标优化 (CMOEA)

一旦转换条件触发，算法进入第二阶段。

- 恢复目标：丢弃 $f'(x)$ ，重新激活原始的 m 个目标函数。
- 算法接入：此时可以接入任何主流的约束多目标进化算法（如 NSGA-II-CDP, MOEA/D-CDP 等）。
- 优势：由于种群已经位于 PF 中心附近的可行区域，CMOEA 只需要进行少量的多样性维护工作，就能迅速覆盖整个 Pareto 前沿。

3.4 伪代码

```
1  算法: ToP (Two-Phase Framework)
2  -----
3  输入: CMOP问题, 种群大小 N, 最大评价次数 Max_FE
4  输出: 最终可行解集 Pt+1
5
6  1. 初始化阶段:
7     - 随机生成大小为 N 的种群 P0
8     - 设置当前阶段标志 Phase = 1
9     - 评价计数 t = 0
10
11 2. 循环演化 (while t < Max_FE):
12
13     IF Phase == 1 (第一阶段):
14         a. 将多目标转化为单目标:  $\min f'(x) = \sum f_i(x)$ 
15         b. 使用 DE 引擎产生子代, 利用“可行性规则”进行环境选择
16         c. 检查转换条件:
17             - 计算可行解比例 Pf
18             - 计算前 1/3 可行解的收敛因子  $\delta$ 
19             - IF (Pf > 1/3 且  $\delta < 0.2$ ):
20                 Phase = 2 (切换到第二阶段)
21
22     ELSE (第二阶段):
23         a. 恢复原始的 m 个目标函数 fi(x)
24         b. 执行标准的约束多目标优化算法 (如 NSGA-II-CDP)
25         c. 进行非支配排序和拥挤度 (或分布性) 计算, 保留优秀个体
26
```

27	$t = t + N$ (更新评价次数)
28	
29	3. 返回最终种群 P_{t+1} 中的非支配可行解
30	-----

Fan Z, Li W, Cai X, et al. Push and pull search for solving constrained multi-objective optimization problems[J]. Swarm and evolutionary computation, 2019, 44: 665-679.

Tian Y, Zhang T, Xiao J, et al. A coevolutionary framework for constrained multiobjective optimization problems[J]. IEEE Transactions on Evolutionary Computation, 2020, 25(1): 102-116.

Ma H, Wei H, Tian Y, et al. A multi-stage evolutionary algorithm for multi-objective optimization with complex constraints[J]. Information Sciences, 2021, 560: 68-91.

算法2（选择算法）的必要性

1 边界未知：没有 Pareto 前沿，加权和根本无法进行科学的“归一化”

审稿人认为“可以直接用期望权重做单目标加权”，这在实际中行不通。因为可靠性是概率（0~1），延迟是绝对时间（毫秒），两者的量纲完全不同，不能直接相加。

要相加就必须归一化，而归一化需要知道当前任务在当前网络状态下的最大值（ T_{max} ）和最小值（ T_{min} ）。在 NP-hard 问题中，不求解无法知道当前任务能达到的最小延迟是多少。

算法先用 RDODA 跑出 Pareto 前沿，探测出了当前真实的可行边界（ $I_{max}, I_{min}, T_{max}, T_{min}$ ），然后再用 DSSA 计算出科学的相对权重（ I_w, T_w ）。如果一开始就用单目标加权，相当于在不知道真实边界的情况下瞎赋权重，权重无法被设置为一个合理的值。

审稿人假设“用户的权重是已知的（weights are already known）”，但论文里的前提是“用户的期望是已知的”（如目标延迟 T_{limit} 、目标可靠性 R_i^{des} ）。

期望值只是一个绝对指标，直接把它转化成固定的加权系数是非常困难的。DSSA 算法（Algorithm 2）精妙之处就在于：它是通过对比用户的期望值与 Pareto 前沿的实际边界，动态计算出权重（Equation 18），从而在实际能做到的方案库里，挑出最贴近用户期望的那个。

2 非凸 Pareto 前沿的捕获失败

我们的模型是一个包含 0-1 部署决策变量（ $x_{i,j}^n$ ）的混合整数规划（MILP），这是一个典型的离散且非凸（Non-convex）的解空间。

在运筹学理论中有一个确凿的结论：简单的线性加权和（Linear Weighted Sum）绝对无法找到非凸 Pareto 前沿上位于凹面（Concave portions）的最优解，也即，线性加权单目标优化只能求解凸包（Convex Hull）上的点，这意味着，一上来就给定权重做单目标优化，即使赋予了完美的权重，算法也将大概率陷入极端解，数学上必定会丢失大量位于非凸前沿凹面（Concave portions）的高质量边云协同策略。而我们必须先用基于支配关系的 NSGA-II 算出完整的 Pareto 前沿，把这些隐藏的优质解全部找出来，最后再用 DSSA 算法进行挑选，这样才能找到合适的解。

补充：根据 Das 和 Dennis (1997) 的经典论证，线性加权和法在处理非凸区域的 Pareto 前沿时会发生几何失效，在线性加权中，目标函数在几何上表现为斜率为权重之比的直线。优化过程实际上是在寻找该直线与可行域边界的切点。如果 Pareto 前沿存在凹陷部分（即非凸区域），无论如何调整权重，切点永远只会落在凹陷区域的两个端点上，而凹陷内部的所有 Pareto 最优解都会被算法忽略。在边缘计算的资源调度中，由于离散变量（如任务执行节点

的二进制选择和非线性约束的存在，可行域往往表现出非凸特性。因此，先验加权法极易漏掉那些在时延与可靠性之间达成极佳平衡的“中间解”。

论文论据：

Das I, Dennis J E. A closer look at drawbacks of minimizing weighted sums of objectives for Pareto set generation in multicriteria optimization problems[J]. Structural optimization, 1997, 14(1): 63-69.

如果多目标问题的 Pareto 前沿是非凸的（存在凹陷部分），那么无论你怎么穷举权重组合（哪怕你把从 0 到 1 的无限个权重都试一遍），“线性加权和”方法永远只能找到边界“凸包”上的点。那些位于凹陷处（Concave portions）的解，在加权求和的单目标目标函数下，它们在数学上永远不可能是极小值点。

3 主观认知偏差

评审人假设“用户的预期权重已经已知”。但在实际工程应用中，用户对“10ms 时延”与“0.01% 可靠性提升”之间的权重衡量往往是模糊的且带有主观偏见。

通过计算 Pareto 前沿并展示不同部署方案的实际性能表现，可以辅助决策者进行“偏好演化”。这种交互式的决策过程比在真空状态下盲目设定权重更具科学性。论文中提出的 DSSA 算法正是基于这种逻辑：它并非在优化开始前强加权重，而是在获得所有最优候选方案后，根据当前应用场景的“期望值”进行理性的二次筛选。